
Can Macros Place Themselves?

Decentralized Agents and Leaderless Swaps

for Macro Placement Refinement

Paulin Renault

Code and data: the `multiagent/` directory of the `placax` repository

Abstract

Learning-based macro placement usually trains a single policy that places or moves one macro at a time. We test the opposite design. Every macro is an agent, all agents run the same small network on their own local view, all macros move at the same time, and they are scored only on the wirelength and overlap of the whole design. This score is differentiable, so each agent's move gets its exact share of the credit by back-propagation. We refine a greedy placement of 128 macros on three benchmarks. On the design they train on, the agents improve wirelength by +7.0%, more than gradient descent on the same objective (+2.0%), but less than gradient descent with its best setting (+16.7%) and less than a one-second search that swaps pairs of macros (+18.1%). They do not transfer to unseen designs. We then let the agents swap: two macros of the same size trade places when each picks the other, and a swap that shares wires with a better one waits. This swap swarm has no coordinator and matches the central swap search, slightly exceeding it on the densest design (+35.3% vs. +33.8%). A swap rule learned from the agents' view does not transfer. The best result on the training design, +25.8%, uses the agents' moves followed by swaps. Against simulated annealing, the classic method built on the same exchange move, the swarm is ahead at equal wall clock (seconds) and behind when annealing is given two orders of magnitude more time: on the densest design annealing reaches +45.4% in 600 s against the swarm's +35.3% in 3.5 s. Two evaluation pitfalls, agents that exploit the legalizer and a legalizer that fails on dense designs, had to be fixed before these comparisons were meaningful.

1 Introduction

Macro placement decides where the large blocks of a chip (memories, IP cores) go, so that the wires between them are short and no two blocks overlap. Reinforcement-learning approaches [6, 7, 11] train one policy that places the macros one after another. Their advantage over simple heuristics is disputed. Placing each macro greedily where it adds the least wire, then searching over the result, does better on standard benchmarks [16], and an independent evaluation found learned placement no better than established methods [2].

We study a decentralized alternative. At each step every macro looks at its surroundings and picks a small displacement, and all macros move together. They share one objective, the wirelength of the whole design plus an overlap penalty, and no macro is rewarded for its own wires. All macros use the same small network, so the rule works for any number of macros and could be applied to a design it was not trained on. We ask four questions.

- Q1** Can agents that act on local information, with one shared policy and one shared objective, improve a placement?
- Q2** How much does each macro need to see: itself, its most connected macros, or a summary of all connected macros?
- Q3** Do the agents beat gradient descent run directly on the macro positions with the same objective?
- Q4** Does a policy trained on one design improve another design without retraining?

Contributions.

- A decentralized refinement method: one policy shared by all macros, simultaneous bounded moves, trained by differentiating the placement objective through the episode (§3).
- An evaluation protocol with controls: a random-noise band, an untrained rule, a swap search, and results over several seeds. It exposed two legalization problems, and we give a legalizer that fixes the second one (§4, §5.1, §5.4).
- Answers to Q1–Q4 on three designs (§5).
- A swap swarm, in which same-size macros exchange positions by mutual consent with a local conflict rule, and which matches a central swap search (§5.6).

2 Related Work

Learning-based macro placement. Mirhoseini et al. [11] place macros one at a time with a graph neural network trained by reinforcement learning. MaskPlace [6] gives the policy pixel maps of legal positions and of the wirelength each position would add. ChiPFormer [7] pre-trains a policy offline on several designs to transfer it. Shi et al. [16] use MaskPlace’s wirelength map greedily and optimize over it with black-box search, which beats these policies. Cheng et al. [2] compared learned macro placement with simulated annealing and commercial tools in an open evaluation and found no advantage. All of these methods move one macro per step.

Analytical and force-directed placement. Force-directed placers [3, 17] treat nets as springs that pull connected cells together and density as a force that spreads them. ePlace [9] and DREAM-Place [8] minimize a smoothed wirelength plus a density term with gradient methods and legalize afterwards. Our Adam baseline is a simple method of this kind.

Swaps and annealing. Exchanging two cells is a standard move in placement, and simulated annealing [15] is the classic method built on it: the same move, accepted sometimes even when it lengthens wires, under a cooling schedule. We use both a greedy swap search and a tuned annealer as baselines, and the same exchange as the move the agents make.

Multi-agent learning and differentiable simulation. When agents share one reward, it is hard to tell which agent caused a change, the credit assignment problem [4]. A differentiable simulator avoids it, because the gradient of the objective gives each agent’s contribution directly. We use the short-horizon training window of SHAC [18] without its value function. A network applied to every parameter with shared weights also resembles a learned optimizer [1], except that our agents see positions, not gradients.

Swarms. Boids [14] produce flocking from three local rules: separation, alignment and cohesion. Neural cellular automata [12] train a local update rule shared by identical cells by back-propagating through the dynamics, as we do. Our conflict rule for swaps is the local-maximum step of Luby’s parallel independent-set algorithm [10], with swap scores as priorities.

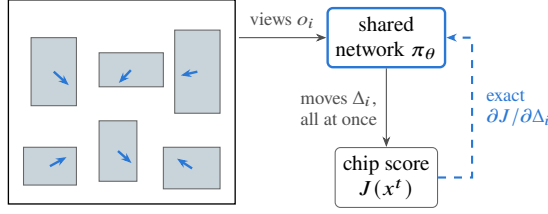
3 Method

3.1 Problem

A design has n macros with lower-left positions $x_i \in \mathbb{R}^2$, measured in grid cells, and fixed sizes. Each net connects pins at fixed offsets on several macros. The half-perimeter wirelength sums, over nets, the width and height of the box around the net’s pins:

$$\text{HPWL}(x) = \sum_{e \in E} \left(\max_{p \in e} p_x - \min_{p \in e} p_x \right) + \left(\max_{p \in e} p_y - \min_{p \in e} p_y \right). \quad (1)$$

(a) Nudging agents



(b) One swap-swarm round

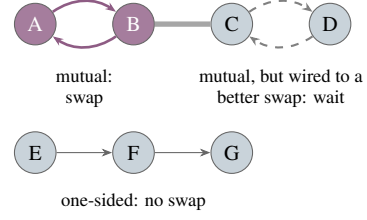


Figure 1: (a) Each macro reads its local view o_i , the shared network turns it into a small move Δ_i , and all macros move at once. The design is scored by one differentiable objective J , and its gradient tells each macro how its own move changed the score. (b) In a round of the swap swarm, each macro names the same-size macro it would most like to swap with. A and B name each other and swap. C and D also agree, but C is wired to B, whose swap scores higher, so C and D wait. E names F but F names G, so nothing happens.

A placement is *legal* if no two macros overlap and all lie on the canvas. Every method starts from the same legal greedy placement x^0 , in which macros are placed one at a time on the free grid cell that adds the least wirelength (MaskPlace’s wire mask used greedily [6, 16]). We report the improvement

$$\text{Impr}(x) = 1 - \frac{\text{HPWL}(\mathcal{L}(x))}{\text{HPWL}(x^0)}, \quad (2)$$

where $\mathcal{L}$ is the legalizer of §3.4. Higher is better. The greedy start has improvement 0, and a negative value means the method made the placement worse.

3.2 Agents

An episode has $T = 32$ steps. At each step every macro i computes its view $o_i(x^t)$ and all macros move together:

$$x_i^{t+1} = \text{clip}(x_i^t + \Delta_i^t), \quad \Delta_i^t = s \tanh(\mu_\theta(o_i(x^t)) + \sigma_\theta \epsilon_i^t), \quad \epsilon_i^t \sim \mathcal{N}(0, I). \quad (3)$$

μ_θ is a two-layer network (64 tanh units per layer) shared by all macros, $s = 1$ grid cell limits each move, and the clip keeps macros on the canvas. The noise ϵ is used in training only. We compare three views, all given relative to the macro and scaled by the canvas size so that one network can run on any design: **m0**, the macro’s position, size and distance to each canvas edge (8 numbers); **m1**, m0 plus the relative position, size and connection weight of its 4 most strongly connected macros (32 numbers); **m1all**, m1 plus the weighted mean and spread of the directions to *all* connected macros, their total connection weight and their count (38 numbers).

3.3 Objective and training

All agents share the objective

$$J(x) = \frac{W_\gamma(x)}{W_\gamma(x^0)} + \lambda_d \frac{D(x)}{A} + \lambda_o \frac{\sum_{i < j} \text{area}(i \cap j)}{A}, \quad (4)$$

where W_γ is a smooth (log-sum-exp) version of (1) with $\gamma = 1$ cell, D is the area by which grid bins exceed 95% density plus the area outside the canvas, A is the total macro area, $\lambda_d = 1$, and λ_o weights the overlap between pairs of macros. The training loss is the mean of J over the $T+1$ placements of an episode. We back-propagate it through (3) and cut the gradient every $H = 8$ steps, as in SHAC [18] but without a value function. Each Δ_i receives the exact derivative of the shared objective. A training run has 100 episodes, uses Adam [5] with learning rate 10^{-3} , and takes about 30 s on a laptop GPU.

Algorithm 1 One round of the swap swarm. Every step runs on all macros in parallel.

Require: placement x ; candidates per macro k ; judge g ; threshold τ

```

1: for each macro  $i$  in parallel do
2:    $C_i \leftarrow$  the  $k$  nearest macros with the same footprint as  $i$ 
3:    $j_i^* \leftarrow \arg \max_{j \in C_i} g_{ij}(x)$ ; if  $g_{ij_i^*}(x) \leq \tau$  then  $j_i^* \leftarrow \emptyset$ 
4: end for
5:  $P \leftarrow \{(i, j) : j_i^* = j \wedge j_j^* = i\}$  ▷ mutual consent
6: for each  $(i, j) \in P$  in parallel do
7:   keep  $(i, j)$  unless an  $(a, b) \in P$  wired to  $\{i, j\}$  has a higher score  $g_{ab} + g_{ba}$  ▷ ties by index
8: end for
9: exchange the positions of every kept pair, simultaneously

```

3.4 Legalization

Gradient-based methods can end with overlapping macros, so every result goes through the same legalizer $\mathcal{L}$ before it is scored. The *repair* step rounds positions to the grid, then takes macros from largest to smallest and moves each overlapping one to the free position, among its 256 nearest, that adds the least wirelength. The *spread* step runs 300 steps of gradient descent on the overlap area alone, moving each macro at most 0.25 cells per step, so that neighbours make room instead of one macro jumping far. $\mathcal{L}$ runs the repair once with and once without spreading and keeps the legal result with the lower HPWL. A placement that is already legal is not changed. Section 5.4 explains why the spread step is needed.

3.5 Swap swarm

Exchanging two macros of the same size keeps a placement legal. Algorithm 1 describes one round. Each macro scores a swap with each of its k nearest same-size macros and names the best. A swap is agreed when two macros name each other. Two agreed swaps that share a wire were each scored as if the other did not happen, so a swap only goes ahead if its score is higher than that of every agreed swap it shares a wire with. Every decision uses only a macro’s candidates and the macros it is wired to. We test two ways to score a swap. The *exact* score g_{ij} is the change in HPWL of the nets on i and j . The *learned* score $\hat{g}_\phi(o_i, o_j, x_j - x_i)$ comes from a network the size of the policy that sees only the agents’ view. It is trained on adaptec1 to predict g_{ij} (Appendix B).

4 Experimental Setup

Designs. Table 1 lists the three designs. The main difference between them is how full the canvas is: bigblue1 is almost empty, and identical SRAM blocks cover half of ariane133.

Baselines and controls. **Adam** [5] minimizes (4) directly over the 256 coordinates for 2000 steps (learning rate 0.05), with the same clip and legalizer. It answers Q3. The **pull rule** has no learning: every step, each macro moves one cell toward the weighted centre of its 4 most connected macros. It shows what an obvious local rule achieves. The **swap search** repeatedly makes the single same-size swap that reduces HPWL the most, until no swap helps. It shows what small continuous moves cannot

Table 1: Designs. On each design, the first 128 macros are placed on a 224×224 grid. Policies are trained on adaptec1. bigblue1 and ariane133 are used only to test transfer.

Design	Source	Macros placed	Canvas covered	Macro shapes
adaptec1	ISPD 2005 [13]	128	28.6%	varied
bigblue1	ISPD 2005 [13]	128	3.5%	thin, varied
ariane133	Ariane RISC-V [2, 19]	128	50.0%	128 identical SRAMs

reach. **Random jitter** adds Gaussian noise with a standard deviation of one cell to the greedy start and legalizes it. It shows how much the legalizer alone can change the result. **Simulated annealing** starts from the same greedy placement and proposes, with equal probability, a swap of two equal-footprint macros or a shift of one macro into free space within a shrinking window. Both moves keep the placement legal, so annealing is scored on its own output with no legalizer. The temperature is steered to a target acceptance rate that falls from 30% to 1%, and the last 10% of the budget runs at zero temperature from the best placement found. We tuned the acceptance target, the share of swaps and the length of that tail on adaptec1 at a 30 s budget (12 configurations, best kept), then used one setting on all three designs. Annealing answers the question a reader asks first, because it makes the same move as our swarm.

Reporting. Every number in this paper is the improvement (2) of a legalized placement over the greedy start of the same design, in percent. Methods without randomness (pull rule, swap search, swap swarm with the exact score) are run once. Adam is run once and evaluated every 100 steps. Each learned policy is trained with 3 seeds and evaluated every 5 episodes. For Adam and for each seed, we report the median of the last 5 evaluations, never the best one, and tables give the mean and standard deviation over the 3 seeds. Random jitter uses 5 noise seeds. *Transfer* runs the adaptec1 policy for one episode, without noise or retraining, on another design. All values are computed from the stored runs by one script (Appendix C).

5 Results

Table 2 gives the main results. The rest of this section explains them.

5.1 Agents exploit the legalizer unless overlap is penalized

We first trained the agents with the density term only ($\lambda_o = 0$). They learned to push macros onto each other and let the legalizer find legal positions (Fig. 2, top). One evaluation of one seed reached +23.6%, but the median over the last evaluations, averaged over three seeds, is +6.5%, and the three seeds differ by up to 27 points. The legalized result also jumps from one evaluation to the next (Fig. 3b), so a single best evaluation says little. With an overlap weight of $\lambda_o = 3$, overlap stays near 3%, the result is stable, and the three seeds give +6.8% with a standard deviation of 0.2 points (Table 3). A weight of 10 stops the agents from moving. We use $\lambda_o = 3$ from here on.

Table 2: Main results: improvement (2) in %, higher is better. Learned methods: mean $\pm$ std over 3 seeds. Jitter: mean $\pm$ std over 5 noise seeds. Other methods are single runs. Best value per design in bold. [†]On bigblue1 and ariane133, the policy trained on adaptec1 is applied without retraining. *One policy seed.

Method	adaptec1 (training)	bigblue1 (unseen)	ariane133 (unseen)
<i>No learning</i>			
Random jitter + legalize (control)	-2.6 $\pm$ 2.5	-1.6 $\pm$ 7.0	-2.1 $\pm$ 0.1
Pull toward partners	+11.0	+15.5	+2.7
Swap search (central)	+18.1	+44.4	+33.8
<i>Direct optimization</i>			
Adam, $\lambda_o = 0$	+16.7	+8.2	+0.6
Adam, $\lambda_o = 3$	+2.0	+4.1	-1.6
<i>Decentralized agents (ours)</i>			
Nudging agents, m1all, $\lambda_o = 3$ [†]	+7.0 $\pm$ 0.1	+1.9 $\pm$ 1.4	-6.6 $\pm$ 2.9
Swap swarm, exact judge	+18.1	+44.4	+35.3
Nudging agents, then swap swarm ^{†*}	+25.8	+38.8	+27.8

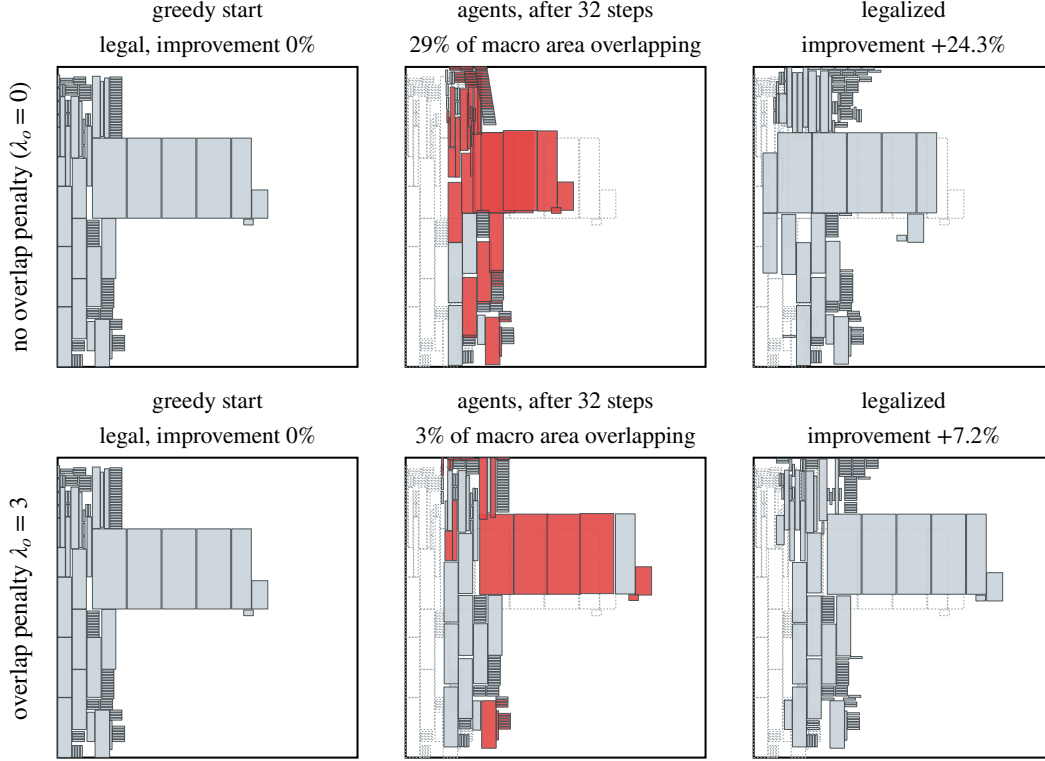


Figure 2: adaptec1, placements produced by the best evaluation of seed 0 (so the top-right value is higher than the median reported in the text). Red macros overlap at least one other macro, even slightly; dashed outlines show the greedy start. **Top:** m1 view without an overlap penalty; the agents pile macros on top of each other and the legalizer rebuilds the placement. **Bottom:** m1all view with $\lambda_o = 3$; the agents make small moves that the legalizer barely changes.

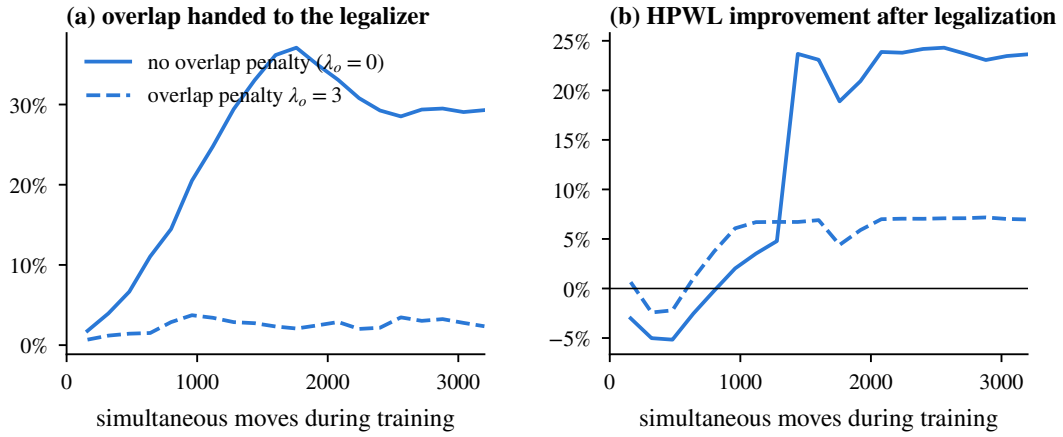


Figure 3: Training on adaptec1 (seed 0, evaluated every 5 episodes; m1 view for $\lambda_o = 0$, m1all view for $\lambda_o = 3$). **(a)** Share of macro area that overlaps before legalization. **(b)** Improvement after legalization. Without the overlap penalty, overlap keeps growing and the result jumps between consecutive evaluations.

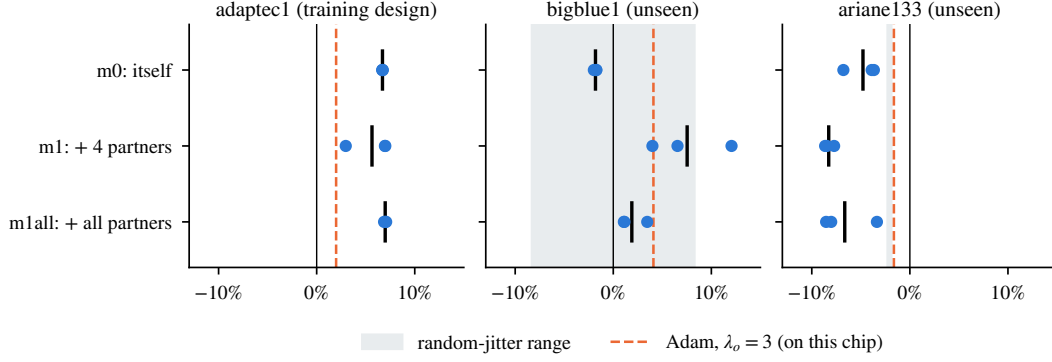


Figure 4: Improvement per view ($\lambda_o = 3$; dots are seeds, bars are means). The orange line is Adam run directly on that design. The grey band is the range of the random-jitter control. A transferred result inside it cannot be told apart from noise.

Table 3: Ablations: improvement in %, mean $\pm$ std over 3 seeds. ‡ Share of macro area overlapping before legalization. § bigblue1 is also a training design here. Policies in the first block were transferred to bigblue1 only.

Variant (m1all unless noted)	adaptec1	overlap ‡	bigblue1	ariane133
<i>Overlap penalty and step schedule</i>				
m1, $\lambda_o = 0$	$+6.5 \pm 14.9$	31%	$+6.8 \pm 8.4$	–
$\lambda_o = 0$	-0.3 ± 11.5	38%	$+4.8 \pm 3.2$	–
$\lambda_o = 3$	$+6.8 \pm 0.2$	3%	$+0.2 \pm 4.4$	–
$\lambda_o = 10$	-0.3 ± 0.3	1%	$+1.9 \pm 1.3$	–
steps 16 $\rightarrow$ 1, $\lambda_o = 0$	$+0.2 \pm 9.9$	63%	$+11.9 \pm 1.3$	–
steps 16 $\rightarrow$ 1, $\lambda_o = 10$	-39.9 ± 5.9	15%	$+5.9 \pm 6.2$	–
<i>Swarm rules and generalization ($\lambda_o = 3$)</i>				
none (reference)	$+7.0 \pm 0.1$	3%	$+1.9 \pm 1.4$	-6.6 ± 2.9
alignment $a = 0.5$	-0.9 ± 0.7	2%	-0.5 ± 2.1	-2.3 ± 0.2
alignment $a = 0.9$	0.0 ± 0.7	1%	$+1.9 \pm 0.7$	-1.6 ± 0.5
random half of macros move	-1.3 ± 1.0	2%	$+8.6 \pm 3.4$	-2.2 ± 0.2
noisy starts ($\sigma = 4$ cells)	$+1.7 \pm 0.3$	9%	$+6.6 \pm 3.7$	-3.1 ± 0.8
noisy starts + trained on bigblue1 §	-1.3 ± 0.9	10%	$+2.8 \pm 3.2$	-2.1 ± 0.7

5.2 Q1, Q2 and Q4: what a macro needs to see, and transfer

Q1. The agents improve their training design by +7.0% with the m1all view, and the three seeds agree to within a fraction of a point.

Q2. On adaptec1 the view hardly matters: m0 reaches +6.7% and m1 +5.6% (Fig. 4). A macro that knows only its own position can still learn where to move on the one design it is trained on. The views differ on other designs. On bigblue1, m0 gives -1.8% and m1 gives +7.5%, so what carries over is knowing where one’s connected macros are.

Q4. That transfer is not larger than noise. Random jitter alone gives up to +8.3% on bigblue1, and on ariane133 every transferred policy makes the placement worse (-8.3% to -1.6%). Two methods from neural cellular automata [12], moving a random half of the macros each step and training from noisy starting placements, reduce overfitting to adaptec1 but do not change this conclusion (Table 3).

5.3 Alignment from swarm models removes the gain

Our objective already contains two of the three boids rules [14]. The wirelength term pulls connected macros together (cohesion) and the overlap penalty keeps them apart (separation). We added the third, alignment: a macro’s move becomes $(1 - a)\Delta_i$ plus a times the weighted mean of its connected macros’ moves. With $a = 0.9$ the improvement on adaptec1 drops to about zero (+0.0%, Table 3).

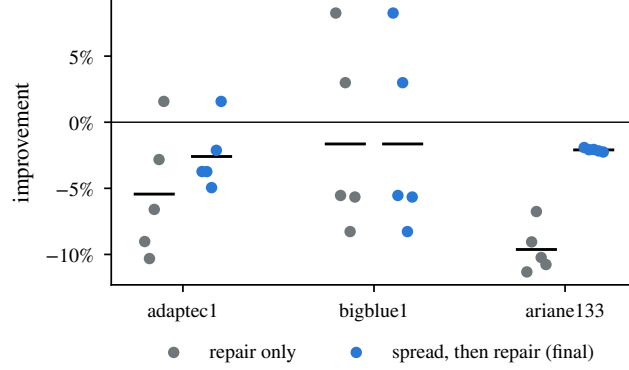


Figure 5: Random-jitter control: the greedy start plus noise of one cell, legalized (5 noise seeds; bars are means). With the repair alone, ariane133 loses about 10%. Spreading first reduces the loss to about 2%. bigblue1 is unchanged, because there the final legalizer always keeps the repair-only result.

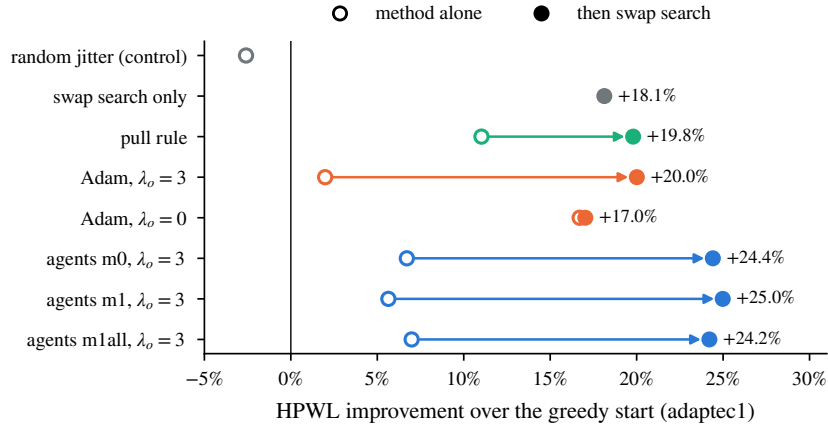


Figure 6: adaptec1: each method alone (hollow circle) and followed by the swap search (filled circle). Learned methods are means over 3 seeds; the swap search is applied to the last evaluated placement of each seed.

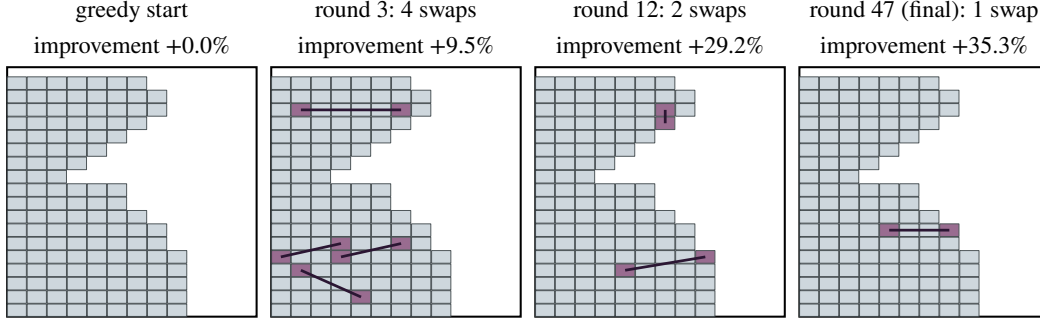
Wirelength only falls when connected macros move *relative* to each other, and alignment makes them move together, so the wires inside a group keep their length. Alignment could help a group travel far across the canvas, but moves of at most one cell per step do not allow that.

5.4 Why the dense design defeated every method

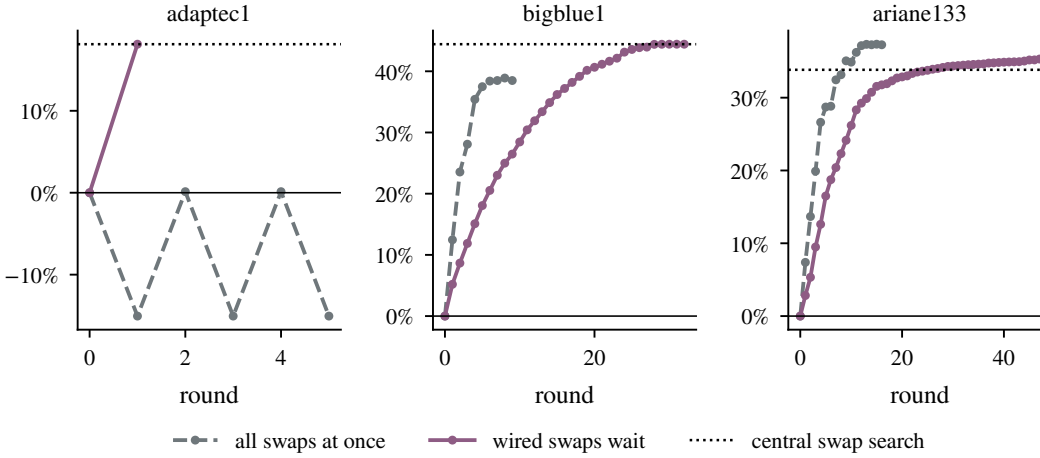
With the repair-only legalizer, every method made ariane133 worse, including Adam, whose placement before legalization was +13.1% better than the start. Identical blocks cover half of ariane133, so the nearest free space is usually far away. After one cell of noise, the repair moved macros 28 cells on average and lost -9.6% (Fig. 5). Adding the spread step reduces this to 2 cells and -2.1%, and Adam on ariane133 then reaches +0.6%. All results in this paper use the final legalizer. Even with it, gradient-based methods gain almost nothing on ariane133, while the swap search gains +33.8%.

5.5 Q3: agents against direct optimization

With the same objective ($\lambda_o = 3$), the agents do better than Adam: +7.0% against +2.0%. Adam with $\lambda_o = 0$ reaches +16.7%, however, and the swap search reaches +18.1%, both more than the agents. Most of the slack in the greedy start is in *which* macro sits in which slot. Fixing that requires two blocks to trade places, and small continuous moves cannot do so without passing through an overlap that the penalty forbids. The agents' moves and the swaps do combine on the training design. Running the swap search after the agents gives +25.0% (m1), more than any other method followed



(a) Rounds of the swap swarm on ariane133 (exact score, all same-size candidates). Purple macros swapped in that round and are linked to their partner.



(b) Improvement after each round. The dotted line is the central swap search.

Figure 7: The swap swarm. On ariane133 it stops after 47 rounds and 90 swaps. Without the conflict rule, adaptec1’s four large macros, which share wires, swap back and forth, and the design ends at -15.1% . Animations are in the supplementary material.

Table 4: Swap swarm variants: improvement in %. Exact-score rows are single runs. Learned rows: mean $\pm$ std over 3 seeds; the last learned row pools 6 runs (m1 and m1all).

Judge	Variant	adaptec1	bigblue1	ariane133
—	central swap search (reference)	+18.1	+44.4	+33.8
exact	all agreed swaps at once	-15.1	+38.5	+37.3
exact	wired swaps wait (Alg. 1)	+18.1	+44.4	+35.3
exact	wired swaps wait, 8 nearest candidates	+18.1	+27.8	+15.5
learned	view m0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0
learned	view m1	$+12.0 \pm 10.4$	-17.4 ± 19.0	-0.6 ± 0.6
learned	view m1all	$+6.0 \pm 10.4$	-2.2 ± 3.0	$+0.6 \pm 0.6$
learned	m1, m1all, also trained on bigblue1	—	—	$+0.9 \pm 1.1$
exact	after the agents’ nudges	+25.8	+38.8	+27.8

by swaps (at most $+20.0\%$; Fig. 6). On unseen designs, the swap search alone does better than any transferred policy, with or without swaps afterwards.

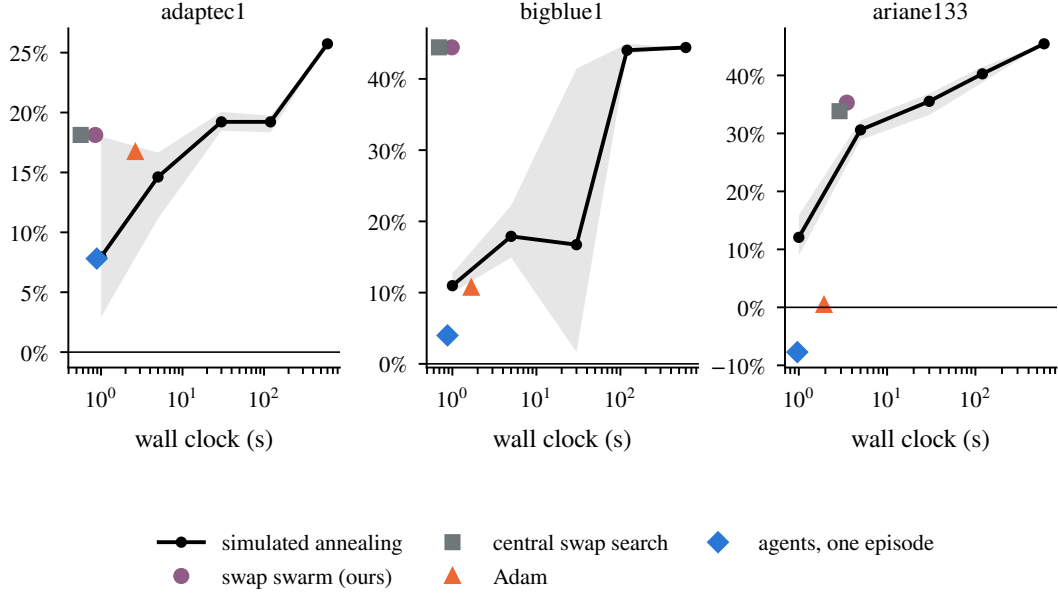


Figure 8: What each method reaches against the time it needs (log scale). The black line is simulated annealing at five budgets, with the band spanning its seeds. Markers are single runs of the other methods at their measured wall clock, including compilation.

Table 5: Simulated annealing by time budget, against our methods at the wall clock they need. Improvement in %, mean $\pm$ std over 3 annealing seeds (1 seed at 600 s).

Method (time budget)	adaptec1	bigblue1	ariane133
<i>Simulated annealing, 3 seeds (600 s: 1 seed)</i>			
1 s	+7.9 $\pm$ 8.7	+11.0 $\pm$ 1.5	+12.1 $\pm$ 3.5
5 s	+14.6 $\pm$ 3.0	+17.9 $\pm$ 3.8	+30.6 $\pm$ 1.7
30 s	+19.2 $\pm$ 0.8	+16.7 $\pm$ 21.6	+35.5 $\pm$ 2.1
120 s	+19.2 $\pm$ 0.8	+44.0 $\pm$ 0.8	+40.3 $\pm$ 1.4
600 s	+25.7	+44.4	+45.4
<i>Ours, at the wall clock each one needs</i>			
swap swarm	+18.1 (0.8 s)	+44.4 (1.0 s)	+35.3 (3.5 s)
agents, then swap swarm	+25.8 (1.7 s)	+38.8 (1.9 s)	+27.8 (4.5 s)

5.6 Agents that swap

With the exact score and the conflict rule, the swap swarm matches the central swap search on adaptec1 and bigblue1 and does slightly better on ariane133, +35.3% against +33.8% (Table 4). It also needs fewer rounds (47) than the central search needs sequential swaps. The conflict rule is necessary: without it, swaps that share wires undo each other and adaptec1 ends at -15.1% (Fig. 7b). Candidates must also be chosen widely. With only the 8 nearest same-size macros as candidates, ariane133 reaches +15.5%, because the best exchange partner is rarely next door.

Learning the swap score gives the same picture as learning the moves. With the m0 view, the learned score never produces a useful swap. With m1 or m1all it finds the three swaps that matter on adaptec1 for some seeds, but it does not help on the other designs, even when trained on two of them (ariane133: +0.9%). The best result on adaptec1, +25.8%, comes from running the agents’ moves first and the swap swarm second. On the other designs the agents’ moves lower the final result.

5.7 Cost: annealing and wall clock

Table 5 and Fig. 8 put every method on one time axis, and Table 6 gives the measured wall clock.

Table 6: Wall clock in seconds, including JAX compilation, and the result reached. One machine: an RTX 3050 laptop GPU, with annealing single-threaded on the CPU.

Method	adaptec1		bigblue1		ariane133	
	s	result	s	result	s	result
central swap search	0.6	+18.1	0.7	+44.4	2.9	+33.8
swap swarm (ours)	0.8	+18.1	1.0	+44.4	3.5	+35.3
agents, one episode	0.9	+7.8	0.9	+4.0	1.0	-7.7
pull rule	2.3	+11.0	1.1	+15.5	1.3	+2.7
Adam, 2000 steps	2.6	+16.7	1.7	+10.7	1.9	+0.4
simulated annealing, 30 s	30	+19.2	30	+16.7	30	+35.5
simulated annealing, 600 s	600	+25.7	600	+44.4	600	+45.4
<i>Loading the netlist and building the greedy start, paid by every method</i>						
greedy start	5.1	—	3.2	—	2.6	—

At equal time, the swarm is ahead. Annealing given one second reaches +7.9% on adaptec1, while the swap swarm reaches +18.1% in 0.8 s and the agents followed by the swarm reach +25.8% in 1.7 s. Annealing needs about 30 s to match the swarm on adaptec1 (+19.2%) and about 120 s on bigblue1.

Given far more time, annealing wins. At 600 s it reaches +25.7% on adaptec1, +44.4% on bigblue1 and +45.4% on ariane133, the last well above the swarm’s +35.3%. The swarm stops when no macro wants to trade, which is a local optimum of its move set, while annealing accepts uphill moves and keeps going.

Read the speed comparison with care. The two implementations do not use the same hardware: the swarm scores all candidate swaps in parallel on a GPU, and the annealer is a single-threaded NumPy loop proposing one move at a time. A hardware-independent measure is how many candidate moves each one scores. On ariane133 the swarm scores 764,032 candidates in total, and annealing proposes 428,461 in 120 s, by which point it is already ahead (+40.3% against +35.3%). On adaptec1 the ordering reverses: the swarm needs 6,910 evaluations for +18.1%, two orders of magnitude fewer than annealing spends before passing it. The swarm is thus more efficient per evaluation on the sparse and mixed designs and less efficient on the dense one, and its wall-clock advantage comes from evaluating a whole round at once.

6 Discussion

What the agents learned. On the design they train on, the agents learn a small, repeatable improvement. The view barely matters there, because even m0 can learn the one design. Seeing connected macros is needed for any transfer, but the transfer we measured is no larger than noise. The agents are most useful before a swap search, and less useful as a rule for new designs.

Why swaps work and learned swap scores do not. Each macro can compute the exact effect of a swap from its own nets, and the conflict rule stops wired neighbours from acting on conflicting plans. The learned score only approximates a value that the macro can compute exactly. That it does not transfer matches the result for moves: features learned on one design do not describe another well.

What the annealing comparison changes. The swarm is not a better search than annealing; it is a faster one per unit of wall clock, and it stops earlier. Both use the same exchange move, and annealing’s willingness to accept a worse placement is what lets it pass the swarm given enough time, especially on the dense design where good exchanges are rare and have to be reached through worse intermediate states. The consequence is to give the swarm the same ability: accept a proposed exchange with a probability set by a temperature, keeping mutual consent and the conflict rule, which turns it into a parallel annealer rather than a parallel descent. We have not tested that.

Evaluation. Two early conclusions were wrong because of the legalizer: a first result of +23.6%, and the impression that ariane133 could not be improved. Any placer evaluated after legalization can be misled in the same way. We suggest reporting a random-jitter range, a swap-search baseline, and medians over several seeds and evaluations.

Limitations. We use three designs and 128 macros per design, and measure only wirelength, because the benchmarks include no timing data. Adam is a weaker baseline than a full analytical placer such as DREAMPlace [8]. The timing comparison is bound to our implementations: a compiled, multi-threaded annealer would propose many more moves per second than our NumPy loop, which would narrow or remove the wall-clock advantage we measure. Swaps are limited to macros of the same size. The agents are trained without a value function. Natural next steps are swaps between macros of different sizes, using the agents and swaps after a stronger initial placer, and a comparison with DREAMPlace on more designs.

7 Conclusion

Macros acting as identical agents with local information can improve a placement. Through small moves the gain is modest and does not transfer to other designs. Through swaps agreed between pairs, with a local rule for conflicts, they do as well as a central swap search, and better than simulated annealing at equal wall clock, though annealing overtakes them when given two orders of magnitude more time. In both cases, learned decisions did not generalize, while exactly computed ones did. Controls for the legalizer were needed to reach these conclusions.

References

- [1] Marcin Andrychowicz, Misha Denil, Sergio Gómez, Matthew W. Hoffman, David Pfau, Tom Schaul, Brendan Shillingford, and Nando de Freitas. Learning to learn by gradient descent by gradient descent. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.
- [2] Chung-Kuan Cheng, Andrew B. Kahng, Sayak Kundu, Yucheng Wang, and Zhiang Wang. Assessment of reinforcement learning for macro placement. In *International Symposium on Physical Design (ISPD)*, 2023.
- [3] Hans Eisenmann and Frank M. Johannes. Generic global placement and floorplanning. In *Design Automation Conference (DAC)*, 1998.
- [4] Jakob Foerster, Gregory Farquhar, Triantafyllos Afouras, Nantas Nardelli, and Shimon Whiteson. Counterfactual multi-agent policy gradients. In *AAAI Conference on Artificial Intelligence*, 2018.
- [5] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.
- [6] Yao Lai, Yao Mu, and Ping Luo. MaskPlace: Fast chip placement via reinforced visual representation learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [7] Yao Lai, Jinxin Liu, Zhentao Tang, Bin Wang, Jianye Hao, and Ping Luo. ChiPFormer: Transferable chip placement via offline decision transformer. In *International Conference on Machine Learning (ICML)*, 2023.
- [8] Yibo Lin, Shounak Dhar, Wuxi Li, Haoxing Ren, Bruce Khailany, and David Z. Pan. DREAMPlace: Deep learning toolkit-enabled GPU acceleration for modern VLSI placement. In *Design Automation Conference (DAC)*, 2019.
- [9] Jingwei Lu, Pengwen Chen, Chin-Chih Chang, Lu Sha, Dennis Jen-Hsin Huang, Chin-Chi Teng, and Chung-Kuan Cheng. ePlace: Electrostatics-based placement using fast Fourier transform and Nesterov’s method. *ACM Transactions on Design Automation of Electronic Systems*, 20(2), 2015.

- [10] Michael Luby. A simple parallel algorithm for the maximal independent set problem. *SIAM Journal on Computing*, 15(4), 1986.
- [11] Azalia Mirhoseini, Anna Goldie, Mustafa Yazgan, Joe Wenjie Jiang, Ebrahim Songhori, Shen Wang, Young-Joon Lee, Eric Johnson, Omkar Pathak, Azade Nazi, Jiwoo Pak, Andy Tong, Kavya Srinivasa, William Hang, Emre Tuncer, Quoc V. Le, James Laudon, Richard Ho, Roger Carpenter, and Jeff Dean. A graph placement methodology for fast chip design. *Nature*, 594:207–212, 2021.
- [12] Alexander Mordvintsev, Ettore Randazzo, Eyvind Niklasson, and Michael Levin. Growing neural cellular automata. *Distill*, 2020.
- [13] Gi-Joon Nam, Charles J. Alpert, Paul Villarrubia, Bruce Winter, and Mehmet Yildiz. The ISPD2005 placement contest and benchmark suite. In *International Symposium on Physical Design (ISPD)*, 2005.
- [14] Craig W. Reynolds. Flocks, herds and schools: A distributed behavioral model. In *Proceedings of SIG-GRAPH*, 1987.
- [15] Carl Sechen and Alberto Sangiovanni-Vincentelli. The TimberWolf placement and routing package. *IEEE Journal of Solid-State Circuits*, 20(2), 1985.
- [16] Yunqi Shi, Ke Xue, Lei Song, and Chao Qian. Macro placement by wire-mask-guided black-box optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [17] Peter Spindler, Ulf Schlichtmann, and Frank M. Johannes. Kraftwerk2—a fast force-directed quadratic placement approach using an accurate net model. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 27(8), 2008.
- [18] Jie Xu, Viktor Makoviychuk, Yashraj Narang, Fabio Ramos, Wojciech Matusik, Animesh Garg, and Miles Macklin. Accelerated policy learning with parallel differentiable simulation. In *International Conference on Learning Representations (ICLR)*, 2022.
- [19] Florian Zaruba and Luca Benini. The cost of application-class processing: Energy and performance analysis of a Linux-ready 1.7-GHz 64-bit RISC-V core in 22-nm FDSOI technology. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 27(11), 2019.

A Hyperparameters

Setting	Value
Grid, macros, canvas	224×224 ; first 128 macros; core area (whole die for ariane133)
Episode	$T = 32$ simultaneous steps, $ \Delta_i \leq 1$ cell per axis
Policy	2 hidden layers of 64 tanh units; initial $\log \sigma = -1.5$
Objective	$\gamma = 1$ cell, target density 0.95, $\lambda_d = 1$, $\lambda_o = 3$ unless stated
Training	100 episodes, Adam, learning rate 10^{-3} , gradient-norm clip 1, window $H = 8$
Evaluation	every 5 episodes, no noise; median of the last 5
Legalizer	repair over the 256 nearest free positions; spread: 300 steps of ≤ 0.25 cells
Adam baseline	2000 steps, learning rate 0.05, evaluated every 100 steps
Swap swarm	$k = 8$ or all same-size macros as candidates; learned-score threshold $\tau = 0.3$

B Learned swap score

The learned score $\hat{g}_\phi$ is a network with two layers of 64 tanh units. Its input is the view of both macros and their relative position. Its target is the exact gain divided by the mean HPWL per macro of the placement, clipped to $[-5, 5]$. Training placements are the greedy start, the greedy start after 3 to 60 random same-size swaps, and the placements visited by the exact swap swarm from these starts. This gives 150,072 candidate pairs on adaptec1, of which 3 to 4% reduce HPWL. We train for 30 epochs with Adam and a Huber loss. At run time a macro proposes a swap only if the predicted gain exceeds $\tau = 0.3$, because most candidate swaps change nothing and small positive predictions for them are noise.

C Reproducibility

`bash multiagent/run_all.sh` reruns every experiment in about 45 minutes on a laptop GPU. `python -m multiagent.paper.build` then recomputes every figure, table and number in this paper from the stored runs and compiles it. GPU computations are not bit-for-bit deterministic, so a rerun reproduces the conclusions but not every digit. Animations of the agents and the swap swarm are in `multiagent/paper/SUPPLEMENTARY.md`.